



Engenharia de software e modelagem II

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

4. Planejamento do Projeto

Como o software será desenvolvido?

O planejamento define como a equipe organizará o trabalho.

Elementos importantes

- Escopo;
- Cronograma;
- Atividades;
- Prioridades;
- Equipe;
- Recursos;
- Riscos;
- Entregas.

Perguntas fundamentais

- **O que será desenvolvido?**
- **Quando será entregue?**
- **Quem fará?**
- **Quais riscos podem afetar o projeto**

5. Arquitetura de Software

Definindo a estrutura da solução

A arquitetura define as principais decisões estruturais do sistema.

Decisões arquiteturais

- Organização dos componentes;
- Comunicação entre módulos;
- Persistência de dados;
- Integração com sistemas externos;
- Segurança;
- Escalabilidade;
- Disponibilidade.

Exemplos de estilos arquiteturais

- Arquitetura em camadas;
- MVC;
- Hexagonal;
- Clean Architecture;
- Microservices;
- Event-Driven.

6. Projeto Detalhado

Como os componentes serão construídos?

Nesta fase, a arquitetura é detalhada em elementos que poderão ser implementados.

São definidos:

- Classes;
- Interfaces;
- Módulos;
- Componentes;
- Serviços;
- Responsabilidades;
- Dependências.

Princípios importantes

- Alta coesão;
- Baixo acoplamento;
- Encapsulamento;
- Separação de responsabilidades;
- SOLID.

Padrões de Projeto

Strategy

Factory

Observer

Adapter

Facade

7. Implementação

Transformando o projeto em código

Nesta fase, os desenvolvedores implementam a solução definida nas etapas anteriores.

Atividades

- Desenvolvimento de funcionalidades;
- Criação de componentes;
- Integração entre módulos;
- Implementação de regras de negócio;
- Acesso a dados;
- Integração com APIs.

Boas práticas

- Controle de versão;
- Revisão de código;
- Padronização;
- Testes automatizados;
- Commits frequentes.

8. Testes e Garantia da Qualidade

O software funciona como esperado?

Os testes buscam identificar problemas antes que o software chegue aos usuários.

Tipos de testes

- **Testes Unitários**
Verificam pequenas unidades de código.
- **Testes de Integração**
Verificam a comunicação entre componentes.
- **Testes End-to-End**
Validam o sistema de ponta a ponta.

9. Integração, Configuração e Entrega

Como garantir que o software possa ser entregue?

O software precisa ser integrado, empacotado e disponibilizado de forma controlada.

Práticas importantes

- Integração Contínua;
- Builds automatizados;
- Gerência de configuração;
- Controle de versões;
- Variáveis de ambiente;
- Contêineres;
- Pipelines.

Commit

Build

Testes

Análise

Empacotamento

Deploy



10. Operação, Manutenção e Evolução

O software continua depois da entrega

Após a disponibilização do sistema, novas necessidades e problemas surgem.

Atividades

- Correção de defeitos;
- Melhorias;
- Novas funcionalidades;
- Monitoramento;
- Atualizações;
- Refatoração;
- Evolução arquitetural.

Feedback

Novos
Requisitos

Desenvolvi
mento

Testes

Nova
Entrega



UMA HORA DEPOIS



Sobre o que iremos falar ?



- **Fase de projeto orientada a objeto** e sua importância.
- Como aplicar **princípios de design OO** (abstração, encapsulamento, herança, polimorfismo, baixo acoplamento e alta coesão).
- Como **priorizar requisitos** usando o **método MoSCoW** para focar no que é essencial.
- As **etapas do projeto OO**, desde o refinamento da análise até a aplicação de padrões de projeto.
- O uso de **UML e ferramentas de modelagem** para documentar e comunicar o projeto.





Projeto orientado a objeto

Projeto OO traduz o modelo de análise para uma solução técnica.

Análise → O que fazer

Projeto → Como fazer

Entregáveis: Diagramas UML, arquitetura, interfaces.



Princípios Fundamentais



- Abstração
 - Encapsulamento
 - Herança
 - Polimorfismo
 - Baixo acoplamento
 - Alta coesão
 - Reuso e extensibilidade
-





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade

Representar apenas as principais características de um objeto, ocultando detalhes irrelevantes

Aplicação no projeto:

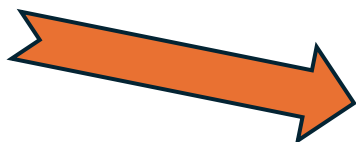
- Criar classes e métodos que **representem conceitos do domínio** sem incluir lógica.
- Exemplo: Em um sistema comercial, criar uma classe Venda com atributos e métodos, sem expor como os cálculos internos são feitos.





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Proteger os dados internos de uma classe, permitindo acesso apenas por Getter's e Setter's

Aplicação no projeto:

- Definir atributos como private e criar métodos get/set.
- Garante que a manipulação dos dados respeite regras de negócio.
- Exemplo: Em Venda, impedir que o estoque seja alterado diretamente, exigindo uso de entrada() ou saída().





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Permitir que uma classe herde atributos e comportamentos de outra.

Aplicação no projeto:

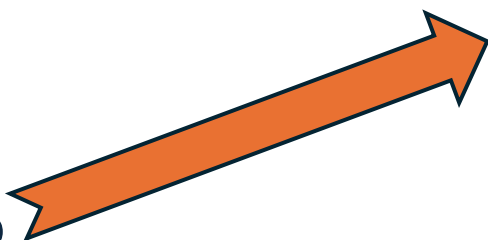
- Criar **classes genéricas** e especializá-las em subclasses, evitando repetição de código.
- Exemplo: Pessoa → Colaborador e Cliente, herdando propriedades comuns e especializando métodos.





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Capacidade de um mesmo método ter comportamentos diferentes dependendo do objeto que o executa.

Aplicação no projeto:

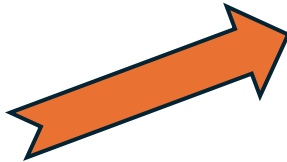
- Criar métodos que possam ser sobrescritos nas subclasses para se adaptar ao contexto.
- Exemplo: Um método `calcularBonus()` implementado de forma diferente para Gerente e Vendedor.



Princípios Fundamentais



- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Minimizar a dependência direta entre classes.

Aplicação no projeto:

- Usar **interfaces e injeção de dependência** para que as classes conheçam apenas contratos, e não implementações específicas.
- Exemplo: Uma classe RelatorioService depender de IExportador (interface), e não diretamente de ExportadorPDF.





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Garantir que cada classe tenha uma única responsabilidade clara.

Aplicação no projeto:

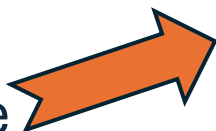
- Evitar que uma classe execute tarefas de diferentes domínios.
- Exemplo: A classe Pedido deve apenas gerenciar informações de pedido, não processar pagamentos.





Princípios Fundamentais

- Abstração
- Encapsulamento
- Herança
- Polimorfismo
- Baixo acoplamento
- Alta coesão
- Reuso e extensibilidade



Criar componentes que possam ser reaproveitados e expandidos sem grande retrabalho.

Aplicação no projeto:

- Criar classes utilitárias, bibliotecas de componentes, padrões de projeto que permitam evolução.
- Exemplo: Criar um módulo de autenticação que possa ser usado em diferentes sistemas da empresa.



Método MoSCoW

- É um método ou ferramenta de priorização usada para:
 - análise de negócios
 - gestão de projetos
 - desenvolvimento de softwares
- Seu objetivo é analisar e classificar a importância de qualquer item, sejam requisitos, funcionalidades, etapas, tarefas ou processos



Método MoSCoW

M – Must have: Essenciais

S – Should have: Importantes

C – Could have: Desejáveis

W – Won't have: Fora do escopo atual



Método MoSCoW

M – Must have: Essenciais

S – Should have: Importantes

C – Could have: Desejáveis

W – Won't have: Fora do escopo atual

Iniciativas que são uma obrigação (“a must” em inglês) para o time. Elas representam necessidades não-negociáveis para o projeto.

- Qualquer coisa nessa categoria é considerada mandatória para o time concluir. Se você está inseguro sobre se algo deve pertencer a esse grupo, pergunte-se o seguinte:
 - o que acontece se finalizarmos sem isso?
 - existe um atalho ou modo mais simples de terminar isso?
 - o projeto vai funcionar sem essa tarefa?



Método MoSCoW

M – Must have: Essenciais

S – Should have: Importantes

C – Could have: Desejáveis

W – Won't have: Fora do escopo atual

- Elas são importantes para o produto final, mas não são vitais.
- Se deixadas de lado, a entrega ainda funciona.
- Mas, quando incluídas, acrescentam um valor significativo ao projeto.



Método MoSCoW

M – Must have: Essenciais

S – Should have: Importantes

C – Could have: Desejáveis

W – Won't have: Fora do escopo atual

- Uma outra forma de descrever essa categoria é a frase “seria legal ter”, pois essas tarefas não são necessárias para o funcionamento do projeto.
- Comparadas com as tarefas Should Have, elas têm um impacto muito menor nos resultados se deixadas de lado.



Método MoSCoW

M – Must have: Essenciais

S – Should have: Importantes

C – Could have: Desejáveis

W – Won't have:

Fora do escopo atual

- Colocar tarefas nesta categoria é uma forma de evitar o *scope creep*, ou seja, o crescimento desorganizado do projeto. Se algo está neste nível, o time sabe que ele não é prioridade para agora, mas até podem vir a ser em um futuro próximo.



Críticos

M



Importantes

S



Desejados

C



Talvez depois

W



Método MoSCoW – Exemplo biblioteca

- **Must have:** Cadastrar usuário, cadastrar livro, empréstimo e devolução
- **Should have:** Renovar empréstimo online
- **Could have:** Avaliar livros
- **Won't have:** Integração redes sociais



Etapas do projeto

1. Refinamento do Modelo de Análise
 2. Definição da Arquitetura
 3. Modelagem de Classes
 4. Modelagem de Interações
 5. Definição de Interfaces
 6. Aplicação de Padrões de Projeto
-



Etapas do projeto

1. Refinamento do Modelo de Análise e requisitos

2. Definição da Arquitetura



Método MoSCoW

The diagram features a dark blue rounded rectangle containing the text 'Método MoSCoW'. A blue arrow originates from the right side of the text '2. Definição da Arquitetura' and points towards the left side of the rectangle. The background of the slide includes a large, light gray circular arc on the right side.

3. Modelagem de Classes

4. Modelagem de Interações

5. Definição de Interfaces

6. Aplicação de Padrões de Projeto

Notações e Ferramentas

- UML: Classe, Sequência, Pacotes entre outros já demonstrados
 - Ferramentas:
 - StarUML,
 - Lucidchart,
 - Visual Paradigm,
 - Draw.io
-



Boas Práticas

- Seguir princípios SOLID
 - Documentar decisões arquiteturais
 - Evitar sobrecarga de responsabilidades
 - Manter consistência entre modelo e código
-

Erros Comuns

- Classes genéricas ou específicas demais
 - Acoplamento excessivo
 - Ignorar requisitos não funcionais
 - Não validar com protótipos
-

